```
package com.tradeflow;

import
org.springframework.web.bind.annotation.*;
import org.springframework.http.HttpStatus;
import
org.springframework.web.bind.annotation.Respo
nseStatus;

import java.time.LocalDateTime;
import java.util.*;

@RestController
@RequestMapping("/api")
public class TradingController {

private final Map<String, Stock> stocks =
new LinkedHashMap<>();
```

```
private final Map<String, Integer> holdings =
new LinkedHashMap<>();

private final List<Trade> history =
new ArrayList<>();

private double cash = 100000.0;

public TradingController() {
addStock("TCS", "Tata Consultancy Services",
3500, 1.24);
addStock("INFY", "Infosys Limited", 1800, 0.82);
addStock("RELIANCE", "Reliance Industries",
2900, -0.36);
addStock("WIPRO", "Wipro Limited", 500, 1.73);
addStock("HDFCBANK", "HDFC Bank", 1680,
0.57);
addStock("ITC", "ITC Limited", 435, -0.21);
}
```

```
private void addStock(
String ticker, String name,
double price, double change) {

stocks.put(
ticker,
new Stock(ticker, name, price, change)
);

holdings.put(ticker, 0);
}

// Display all stocks
@GetMapping("/stocks")
public Collection<Stock> getStocks() {
return stocks.values();
}

// Display account and portfolio
@GetMapping("/account")
```

```
public Map<String, Object> getAccount() {

double portfolioValue = 0;

for (Stock stock : stocks.values()) {
portfolioValue += stock.price()
* holdings.get(stock.ticker());
}

Map<String, Object> account =
new LinkedHashMap<>();

account.put("cash", cash);
account.put("portfolioValue", portfolioValue);
account.put("totalValue", cash + portfolioValue);
account.put("holdings", new
LinkedHashMap<>(holdings));

return account;
}
```

```
// Buy or sell stocks
@PostMapping("/trade")
public synchronized Map<String, Object> trade(
@RequestBody TradeRequest request) {

String ticker = request.ticker();

if (ticker == null || !stocks.containsKey(ticker)) {
throw new IllegalArgumentException(
"Invalid stock symbol."
);
}

int quantity = request.quantity();

if (quantity <= 0) {
throw new IllegalArgumentException(
"Quantity must be positive."
);
```

```
}

Stock stock = stocks.get(ticker);
double total = stock.price() * quantity;

String type = request.type() == null
? ""
: request.type().toUpperCase(Locale.ROOT);

if (type.equals("BUY")) {

if (total > cash) {
throw new IllegalArgumentException(
"Insufficient balance."
);
}

cash -= total;
holdings.put(
ticker,
```

```
holdings.get(ticker) + quantity
);

} else if (type.equals("SELL")) {

if (quantity > holdings.get(ticker)) {
throw new IllegalArgumentException(
"Insufficient shares."
);
}

cash += total;
holdings.put(
ticker,
holdings.get(ticker) - quantity
);

} else {
throw new IllegalArgumentException(
"Order type must be BUY or SELL."
```

```
);
}

Trade trade = new Trade(
LocalDateTime.now().toString(),
type,
ticker,
quantity,
stock.price(),
total,
"COMPLETED"
);

history.add(trade);

Map<String, Object> result =
new LinkedHashMap<>();

result.put("message", "Trade completed
successfully.");
```

```
result.put("trade", trade);
result.put("account", getAccount());

return result;
}

// Show all completed trades
@GetMapping("/history")
public List<Trade> getHistory() {
return new ArrayList<>(history);
}

// Handle invalid requests
@ExceptionHandler(IllegalArgumentException.cl
ass)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public Map<String, String> handleError(
IllegalArgumentException ex) {

return Map.of("error", ex.getMessage());
```

```
}

public record Stock(
String ticker,
String name,
double price,
double changePercent) {}

public record TradeRequest(
String type,
String ticker,
int quantity) {}

public record Trade(
String time,
String type,
String ticker,
int quantity,
double price,
double total,
```

```
String status) {}
}
```